

LOGIN & LOGOUT API TEST REPORT

Project: Smart Parking System – Web MVP

Testing Area: Authentication – Login & Logout API

Tester: Tạ Ngọc Hoàng Phan

Test Environment: Local Development Environment

API Service: SmartParking.UserService.API

Base URL: <http://localhost:5035>

Test Scope: Login and Logout APIs

Test Status: Completed

1. Objective

The objective of this testing activity is to retest the Login and Logout APIs of the Smart Parking System based on the latest API Design and the current implementation available on the demo branch.

The testing focuses on:

- Login with valid and invalid credentials.
- Request validation.
- Authentication failure handling.
- JWT access token behavior.
- Logout and session revocation.
- Token reuse after logout.
- Multiple concurrent sessions.
- Authorization behavior after logout.
- Login response contract.
- Security-related behavior.
- API Design versus actual implementation consistency.

Register and User Management APIs are currently outside the testing scope because the backend implementation for these features is not currently available.

2. Testing Baseline

2.1 API Design Baseline

The latest API Design defines a REST API using JSON and JWT Bearer Token authentication.

The API base path is:

/api

The relevant authentication endpoints are:

Method	Endpoint	Purpose	Authentication
POST	/api/auth/login	Login	Public
POST	/api/auth/logout	Logout and revoke token	Bearer Token

The Login API is specified to authenticate an account using email and password and issue an Access Token and Refresh Token together with personal information.

2.2 Implementation Baseline

The tested implementation was taken from the current demo branch.

Relevant implementation components include:

- AuthController
- LoginCommand
- LoginCommandHandler
- LoginCommandValidator
- LogoutCommand
- AccessTokenService
- IAuthSessionStore
- InMemoryAuthSessionStore

- JWT authentication configuration
- /api/auth/me authorization endpoint

The current implementation uses JWT Access Tokens with a 300-second lifetime.

Session information is maintained using an in-memory session store. Logout revokes the corresponding session identified by the JWT `jti` claim.

3. API Contract Review

3.1 Login Contract

Endpoint

POST /api/auth/login

Request

```
{
  "email": "driver@gmail.com",
  "password": "Password@123"
}
```

Expected Behavior

The API should authenticate the supplied credentials and return session information containing authentication tokens and user information.

Actual Response Structure

The successful response contained:

- success
- message
- data.accessToken
- data.refreshToken
- data.tokenType
- data.expiresIn

- `data.user.userId`
- `data.user.fullName`
- `data.user.email`
- `data.user.role`

The actual Access Token was received successfully.

The Refresh Token field was present but its value was `null`.

4. Logout Contract

4.1 Endpoint

POST `/api/auth/logout`

Authentication

Bearer Access Token is required.

Expected Behavior

A valid authenticated session should be revoked when Logout is requested.

Actual Behavior

The Logout API returned HTTP 204 No Content for a valid active session.

After logout, the revoked Access Token could no longer access the protected `/api/auth/me` endpoint and resulted in HTTP 401 Unauthorized.

This confirms that session revocation is functioning in the current implementation.

5. Test Execution Summary

A total of **19 test cases** were executed.

Result	Count
--------	-------

PASS	19
FAIL	0
Executed	19
Not Executed	1
Not Applicable	1

The executed functional test cases all produced the expected result.

However, one API contract mismatch was identified regarding the Refresh Token.

Therefore, the result should not be interpreted as a completely issue-free implementation.

6. Login Test Results

Test Case	Scenario	Expected Result	Actual Result	Status
LOGIN-001	Valid credentials	HTTP 200 and session information returned	HTTP 200, Access Token and user information returned	PASS
LOGIN-002	Empty email	HTTP 400	HTTP 400	PASS
LOGIN-003	Invalid email format	HTTP 400	HTTP 400	PASS
LOGIN-004	Empty password	HTTP 400	HTTP 400	PASS
LOGIN-005	Wrong password	HTTP 401	HTTP 401	PASS
LOGIN-006	Unknown email	HTTP 401	HTTP 401	PASS
LOGIN-007	Missing email field	HTTP 400	HTTP 400	PASS
LOGIN-008	Missing password field	HTTP 400	HTTP 400	PASS
LOGIN-009	Login response contract	Required response	Access Token and user information returned; Refresh Token was null	PASS with Contract Mismatch

information
returned

Login Validation Summary

The Login API correctly handled valid credentials, invalid credentials, missing fields, and invalid email format.

Both an unknown email and an incorrect password returned HTTP 401, without exposing a different HTTP status for the two cases.

7. Logout and Session Test Results

Test Case	Scenario	Expected Result	Actual Result	Status
LOGOUT-001	Logout with valid token	HTTP 204	HTTP 204	PASS
LOGOUT-002	Reuse revoked token	HTTP 401	HTTP 401	PASS
LOGOUT-003	Logout without Authorization Header	HTTP 401	HTTP 401	PASS
LOGOUT-004	Logout with invalid Bearer token	HTTP 401	HTTP 401	PASS
LOGOUT-005	Double Logout	Second request rejected	HTTP 401	PASS
LOGOUT-006	Login → Logout → Login	New login should succeed	New login returned HTTP 200	PASS
LOGOUT-007	Multiple sessions	Logging out Session A should not invalidate Session B	Session B remained authorized	PASS
LOGOUT-008	Revoked token accesses /me	HTTP 401	HTTP 401	PASS
LOGOUT-009	/me without token	HTTP 401	HTTP 401	PASS
SECURITY-001	Malformed JWT accesses /me	HTTP 401	HTTP 401	PASS

8. Additional Test Cases

8.1 Expired Access Token

Status: NOT EXECUTED

The implementation currently reports an Access Token lifetime of 300 seconds.

The actual login test confirmed:

- Login Status: HTTP 200
- ExpiresIn: 300
- Access Token received successfully

The token was not allowed to naturally expire during the test execution. Therefore, an expired-token test is not marked as PASS or FAIL.

No artificial token modification or implementation change was performed to manufacture an expired token.

8.2 Refresh Token After Logout

Status: NOT APPLICABLE

The current Login implementation returned:

```
refreshToken = null
```

Therefore, there was no actual Refresh Token available to execute a Refresh Token after Logout test.

This limitation is recorded separately as an API Design versus Implementation finding.

9. API Design vs Implementation

Area	API Design	Current Implementation	Assessment
------	------------	------------------------	------------

Login Endpoint	POST /api/auth/login	Implemented	Match
Logout Endpoint	POST /api/auth/logout	Implemented	Match
Authentication	JWT Bearer Token	JWT Bearer Token	Match
Access Token	Required	Returned	Match
Refresh Token	Required by API Design	null	Mismatch
Token Type	Bearer	Bearer	Match
Token Expiration	Defined by implementation	300 seconds	Implemented
User Information	Returned after login	Returned	Match
Logout	Revoke token	Session revoked	Match
Authorization after logout	Revoked token should not remain usable	HTTP 401	Match

10. Findings

F-01 – Refresh Token Is Specified but Not Implemented

Severity: Medium

Category: API Design vs Implementation

Description

The API Design specifies that the Login API issues both an Access Token and a Refresh Token.

The current implementation successfully returns an Access Token, but the Refresh Token value is null.

Evidence

Actual Login execution:

- HTTP Status: 200
- Access Token: Present
- Refresh Token: null

Impact

A client cannot currently perform a Refresh Token-based session renewal using the current implementation.

Recommendation

Confirm with the development/architecture team whether Refresh Token functionality is:

1. Required for the current MVP baseline and should be implemented, or
2. Intentionally deferred for the current demo/MVP version and therefore the API Design should be updated accordingly.

F-02 – Plain-Text Password Comparison in Demo Implementation

Severity: Medium

Category: Security / Implementation Limitation

Description

The current demo implementation compares the supplied password directly with the seeded user's password value.

This behavior is suitable only as a temporary demonstration mechanism and should not be considered production-ready password handling.

Impact

Plain-text password handling would expose credentials to unnecessary security risks if used in a production environment.

Recommendation

Use a secure password hashing mechanism for production authentication.

This finding is treated as an implementation/security limitation and did not cause the functional Login tests to fail.

F-03 – In-Memory Session Store

Severity: Low / Medium

Category: Implementation Limitation

Description

Authentication session state is currently stored in memory.

The session store supports:

- Session creation
- Session validation
- Session expiration
- Session revocation

However, the session state is not persistent across API restarts.

Impact

Restarting the API can invalidate currently stored authentication sessions.

Recommendation

For production deployment, use an appropriate persistent or distributed session/revocation mechanism if required by the final architecture.

F-04 – Dependency Security Warning

Severity: To Be Confirmed

Category: Build / Dependency

Description

The API build produced an NU1903 warning related to `Microsoft.OpenApi 2.0.0`.

This warning indicates a known high-severity vulnerability associated with the dependency version.

Impact

This is not a functional Login/Logout failure, but it should be reviewed as part of dependency and project security maintenance.

Recommendation

The development team should review the dependency version and determine an appropriate upgrade or mitigation.

11. Security Test Summary

The following authentication and authorization behaviors were verified successfully:

- Invalid credentials are rejected.
- Unknown users are rejected.
- Missing authentication headers are rejected.
- Invalid Bearer tokens are rejected.
- Malformed JWTs are rejected.
- Revoked tokens cannot access protected endpoints.
- Double logout is rejected.
- Logging out one session does not invalidate another active session.
- A user can log in again after logging out.
- Access Tokens have a defined lifetime.

No authentication bypass was identified during the executed tests.

12. Regression Test Summary

The following authentication flow was verified as a regression scenario:

Login → Obtain Access Token → Logout → Re-login

Result:

- Initial Login: PASS
- Logout: PASS
- Revoked token rejected: PASS
- New Login: PASS

Multiple-session behavior was also verified.

Session B remained authorized after Session A was logged out, demonstrating that revocation is session-specific in the current implementation.

13. Overall Assessment

The current Login and Logout implementation is functionally working for the tested scenarios.

All **19 executed test cases passed their expected functional behavior**, and no functional Login/Logout failure was identified.

However, the implementation does not fully match the current API Design because the API Design specifies a Refresh Token while the current Login implementation returns `null` for the Refresh Token.

Additional implementation/security limitations were also identified:

- Plain-text password comparison in the demo implementation.
- In-memory session storage.
- Dependency security warning during build.

The expired-token scenario was not directly executed because the token was not allowed to naturally expire during testing.

14. Final Verdict

PASS WITH MINOR ISSUES

Rationale

- 19 executed functional test cases passed.
- No Login or Logout functional failure was identified.
- Token revocation works correctly.
- Authorization after logout behaves correctly.
- Multiple sessions behave correctly.
- One API contract mismatch was identified: Refresh Token is specified but currently not implemented.
- Additional demo/security limitations were identified and should be addressed or confirmed before production use.

The current Login/Logout implementation is therefore considered **functionally acceptable for the current demo/MVP baseline, with issues requiring confirmation and/or future improvement.**

15. Scope Limitation

The following APIs were not included in this retest:

- Driver Registration
- Owner Registration
- User Management APIs

These APIs are present in the API Design but are currently outside the Login/Logout testing scope because their backend implementation is not currently available in the tested baseline.

No PASS/FAIL result is assigned to those APIs.

16. Evidence Handling

Actual Access Tokens were not included in this report to avoid exposing authentication credentials.

Token values should be represented as:

<REDACTED>

The test results in this report are based on actual API execution against the local `SmartParking.UserService.API` implementation and the latest API Design baseline.